

Modelagem de Sistemas

Técnicas de Modelagem

Ferramentas, Diagramas e Aplicações Práticas

Curso Técnico de Desenvolvimento de Sistemas

Prof. Mizaël Carlos

O que é Modelagem de Sistemas?

Modelar é criar uma representação simplificada da realidade para entender, comunicar e construir sistemas.



Abstração

Foca nos aspectos relevantes e ignora detalhes desnecessários. Facilita o raciocínio sobre sistemas complexos.



Comunicação

Cria uma linguagem comum entre analistas, desenvolvedores, clientes e gestores durante o projeto.



Documentação

Registra decisões de projeto. Serve de referência para manutenção, testes e evoluções futuras do sistema.



Modelos não precisam ser perfeitos — precisam ser úteis para quem vai utilizá-los.

Por que Modelar? Benefícios no Desenvolvimento

Principais Benefícios



Reduz ambiguidades nos requisitos antes de codificar



Facilita comunicação entre equipes técnicas e de negócio



Identifica problemas de design antes da implementação



Estima melhor esforço, prazo e custo do projeto



Gera documentação viva alinhada ao sistema real



Serve de base para planejamento de testes

3x

menos bugs
em produção

40
%

menos retrabalho
no código

2x

mais velocidade
no onboarding

UML — Unified Modeling Language

Linguagem padrão definida pela OMG para modelagem orientada a objetos. Versão atual: UML 2.5.

Diagramas Estruturais

- Classes
- Objetos
- Componentes
- Pacotes
- Implantação
- Estrutura Composta
- Perfil

Diagramas Comportamentais

- Casos de Uso
- Sequência
- Comunicação
- Atividades
- Máquina de Estados
- Temporização
- Visão Geral

Diagrama de Classes — Introdução

O diagrama mais utilizado da UML. Representa a estrutura estática do sistema: classes, atributos, métodos e relacionamentos.

Elementos Principais

Classe	Agrupamento de nome, atributos e métodos.
Atributo	Dado armazenado na classe (tipo + visibilidade).
Método/Operação	Comportamento ou função executada pela classe.
Associação	Linha ligando duas classes relacionadas.
Herança	Seta com triângulo: classe filha herda da pai.
Composição	Losango preenchido: parte depende do todo.
Multiplicidade	1, *, 0..1, 1..N — quantidade de instâncias.

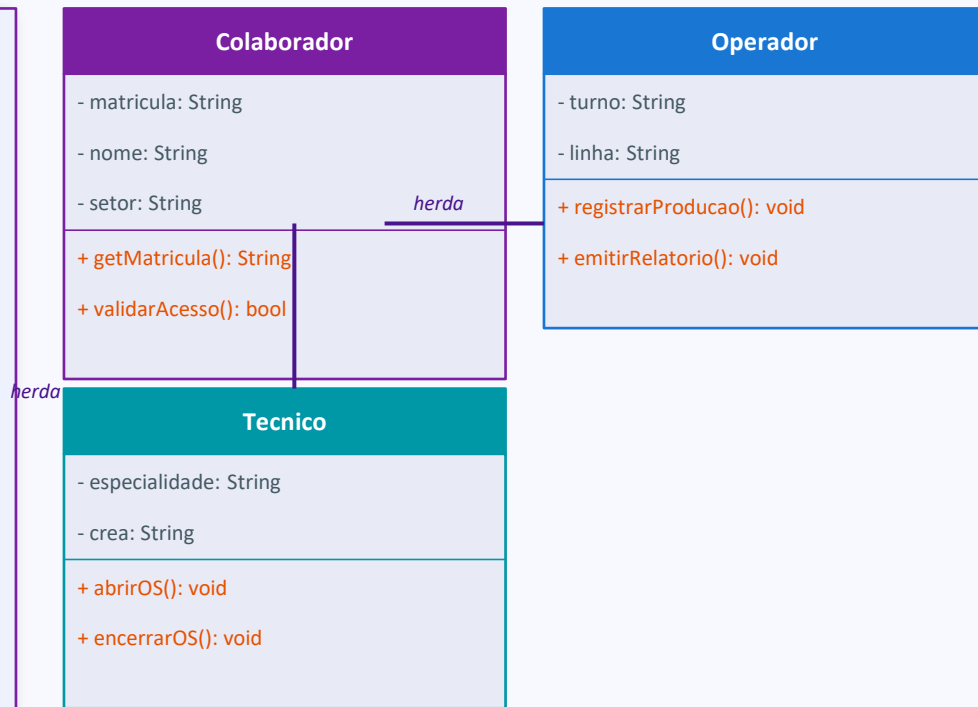


Diagrama de Classes — Visibilidade e Relacionamentos

Cada elemento tem visibilidade definida por símbolos. Os relacionamentos expressam como as classes se conectam.

Visibilidade dos Membros

+ (public)

Acessível por qualquer classe

- (private)

Acessível somente pela própria classe

(protected)

Acessível pela classe e subclasses

~ (package)

Acessível pelo mesmo pacote/módulo

Tipos de Relacionamento

- **Herança** Seta com triângulo vazado — is-a
- **Realização** Seta tracejada com triângulo — implements
- **Associação** Linha simples — usa/conhece

Exemplo: Controle de Manutenção Industrial

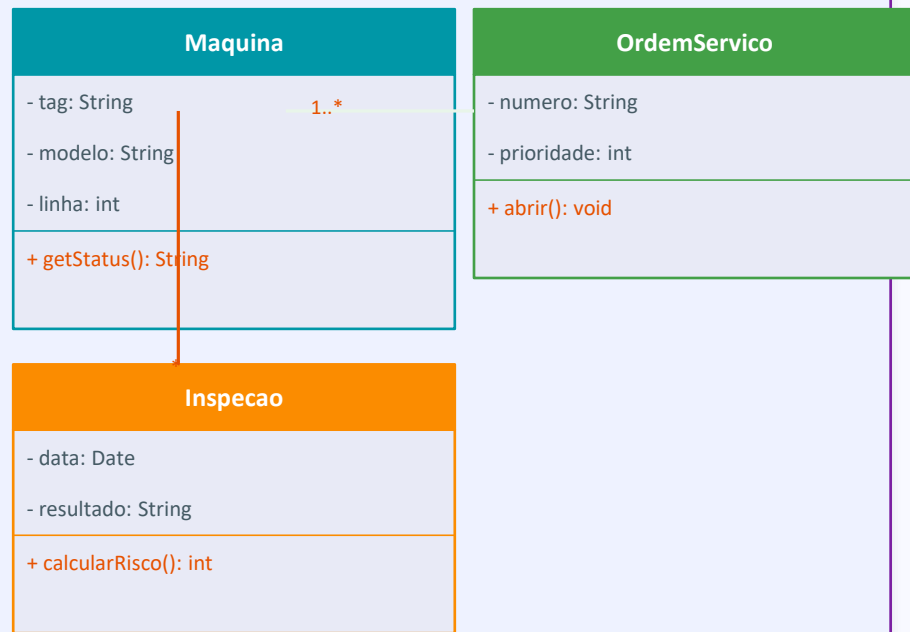


Diagrama de Classes — Herança e Polimorfismo

A herança é a base da reutilização em OO. O polimorfismo permite que métodos se comportem de formas diferentes.

Conceitos Essenciais

Superclasse (Pai)

Classe genérica que define atributos e métodos comuns a subclasses.

Subclasse (Filha)

Herda e especializa a superclasse. Pode sobrescrever métodos.

Sobrescrita (Override)

Subclasse redefine o comportamento de um método herdado.

Classe Abstrata

Não pode ser instanciada. Define contrato para subclasses.

Interface

Define apenas assinaturas de métodos. Nenhuma implementação.

Realização

Classe que implementa uma interface. Representada por seta tracejada.

Hierarquia: Equipamento Industrial

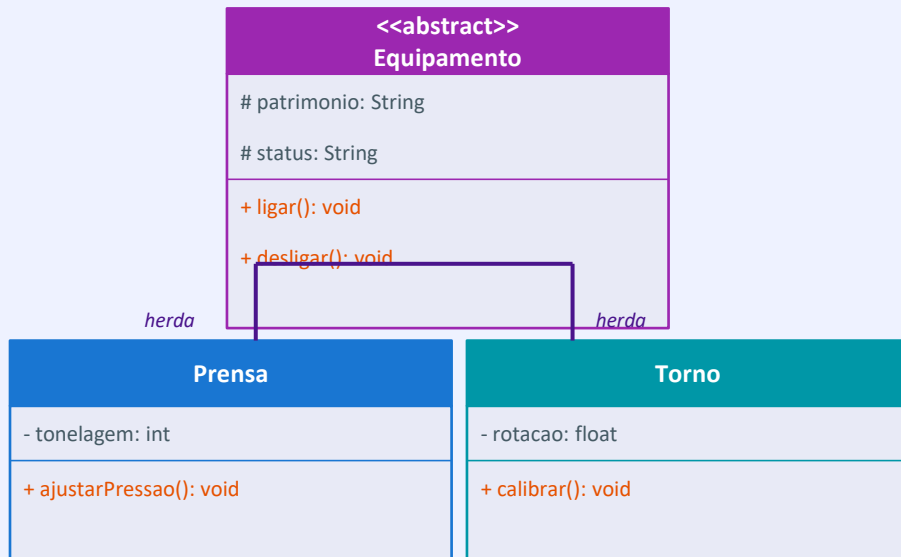


Diagrama de Classes — Exemplo: Sistema de Produção Industrial

Modelo completo: herança, composição e multiplicidade em um sistema de controle de ordens de produção.

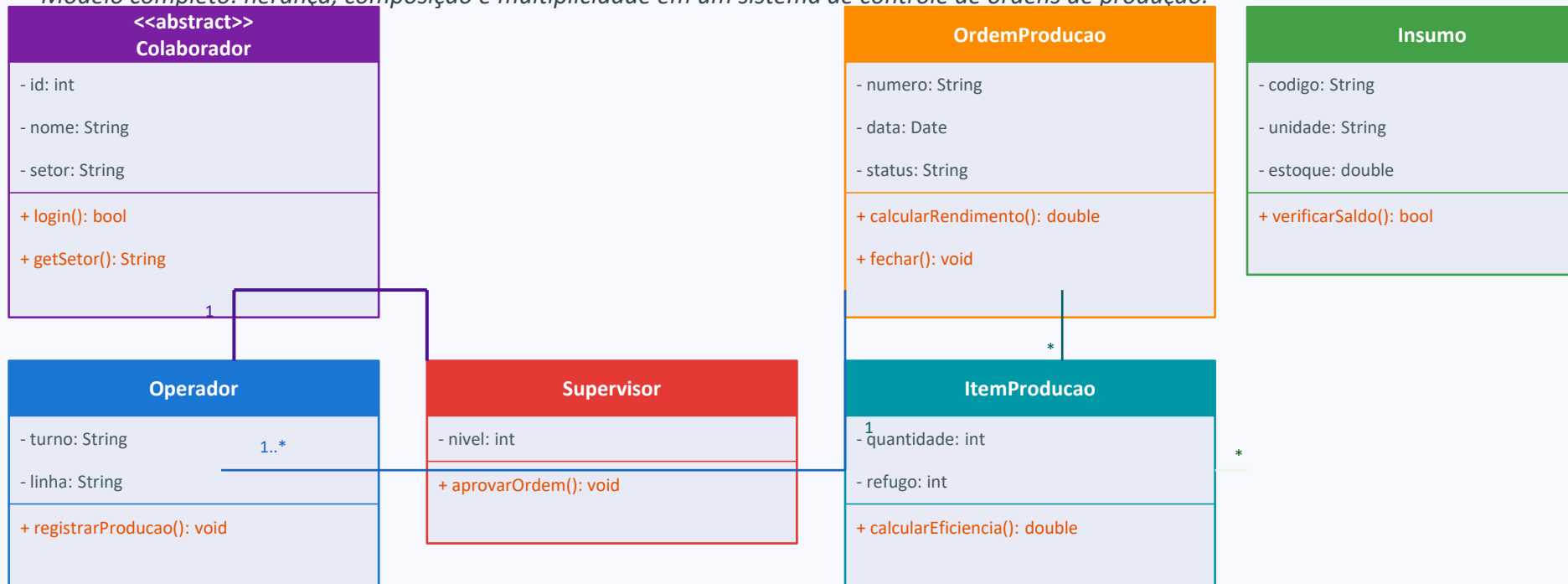


Diagrama de Casos de Uso — Introdução

Representa as funcionalidades do sistema sob a perspectiva do usuário. Foco no QUE o sistema faz, não em COMO.

Elementos do Diagrama

- **Ator**
Entidade externa que interage com o sistema (usuário, sistema externo).
- **Caso de Uso**
Elipse com nome: funcionalidade que agrega valor ao ator.
- **Associação**
Linha entre ator e caso de uso: participação na função.
- **<<include>>**
Caso de uso obrigatoriamente executa outro caso de uso.
- **<<extend>>**
Caso de uso opcionalmente estende outro em condições específicas.
- **Generalização de Ator**
Um ator herda comportamentos de outro ator (seta).
- **Fronteira do Sistema**
Retângulo que delimita o escopo do sistema modelado.

Sistema de Controle de Produção



Casos de Uso — <<include>> e <<extend>>

Esses estereótipos definem dependências entre casos de uso e evitam repetição no modelo.

<<include>> — Inclusão Obrigatória

O caso de uso BASE obrigatoriamente executa o caso de uso INCLUÍDO toda vez que for chamado. Usado para extrair comportamento comum e reutilizável.

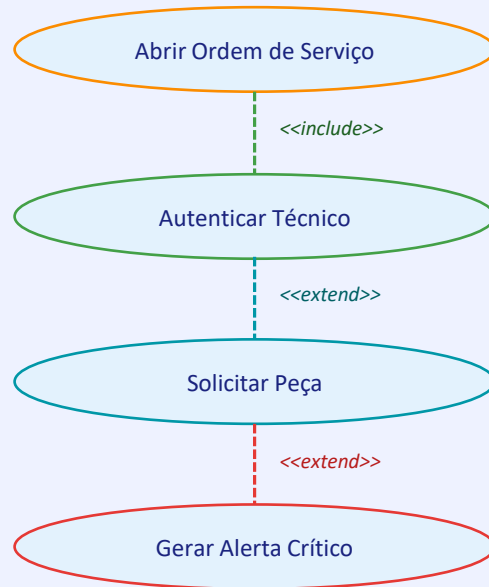
Ex: "Registrar Produção" <<include>> "Autenticar Operador"

<<extend>> — Extensão Opcional

O caso de uso EXTENSOR adiciona comportamento opcional ao caso de uso BASE somente quando uma condição específica é atendida.

Ex: "Gerar Alerta de Parada" <<extend>> "Inspeccionar Máquina" [se crítico]

Sistema: Controle de Manutenção



Casos de Uso — Boas Práticas e Narrativa

Um bom caso de uso precisa de um nome claro (verbo + substantivo) e de uma narrativa estruturada que descreva o fluxo.

Regras de Nomenclatura

✓ Use Verbo + Substantivo: "Realizar Pagamento"

✓ Use perspectiva do ator: "Consultar Extrato"

✓ Seja específico: "Cancelar Pedido" > "Gerenciar"

✗ Evite verbos de sistema: "Processar", "Executar"

✗ Evite casos de uso tecnológicos: "Fazer Query"

✗ Evite nomes genéricos: "Manutenção", "CRUD"

✳ *Cada caso de uso deve representar uma meta completa do ator*

Narrativa do Caso de Uso

Nome: UC-03: Abrir Ordem de Serviço

Ator Principal: Técnico de Manutenção

Pré-condição: Técnico autenticado; máquina cadastrada no sistema

Fluxo Principal:

1. Técnico identifica falha na máquina
2. Acessa o sistema e localiza o equipamento
3. Preenche descrição da falha e prioridade
4. Sistema gera número da OS
5. Sistema notifica supervisor
6. Técnico inicia atendimento

Fluxo Alternativo: 3a. Peça indisponível → sistema abre solicitação de compra

Pós-condição: OS aberta e supervisor notificado automaticamente

Casos de Uso — Exemplo: Sistema de Gestão de Fábrica

Modelo com múltiplos atores, fronteira do sistema, include, extend e generalização de atores



Diagrama de Atividades — Introdução

Representa fluxos de trabalho, processos e algoritmos. Similar ao fluxograma, com notação orientada a objetos.

Elementos do Diagrama

- **Nó Inicial**
Ponto de partida do fluxo de atividade.
- ⊗ **Nó Final**
Encerramento do fluxo principal.
- **Ação / Atividade**
Etapa de trabalho executada por um ator ou sistema.
- ◇ **Decisão / Merge**
Bifurcação condicional baseada em uma guarda [cond].
- **Fork / Join**
Inicia ou sincroniza fluxos paralelos.
- **Fluxo de Controle**
Seta de transição indicando ordem de execução.
- [] **Condição de Guarda**
Expressão booleana sobre a transição de saída.
- || **Raias (Swimlanes)**
Organiza ações por participante/responsável.

Fluxo: Fabricação de Peça Usinada

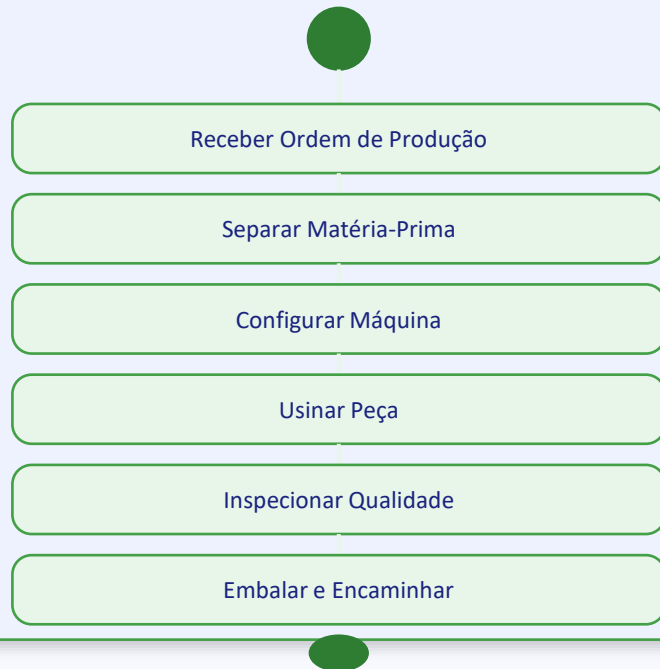


Diagrama de Atividades — Decisão, Fork e Join

Estruturas de controle de fluxo permitem modelar condições, paralelismo e sincronização de processos.

Estruturas de Controle

Nó de Decisão (◇)

Saída única baseada em condição. Cada seta tem uma guarda [cond].

Nó de Merge (◇)

Reunifica fluxos alternativos. Apenas um fluxo chega por vez.

Fork (—)

Divide o fluxo em dois ou mais fluxos concorrentes/paralelos.

Join (—)

Sincroniza fluxos paralelos — espera todos chegarem para continuar.

Região Interrompível

Área que pode ser interrompida por um evento de exceção.

Nó de Objeto

Representa dados sendo passados entre ações (caixa retangular)

Fluxo com Fork/Join — Montagem de Subconjunto



Diagrama de Atividades — Swimlanes (Raias)

Raias (Swimlanes) organizam ações por responsável, tornando explícito quem executa cada etapa do processo.

Características das Raias

- Cada raia representa um ator, departamento ou sistema.
- As ações são posicionadas na raia do responsável.
- O fluxo pode cruzar raias, indicando transferência de responsabilidade.
- Raias podem ser horizontais (piscina horizontal) ou verticais.
- Muito úteis para modelagem de processos de negócio (BPMN também usa).
- Facilitam a identificação de gargalos e responsabilidades.

Processo de Liberação de Manutenção Corretiva

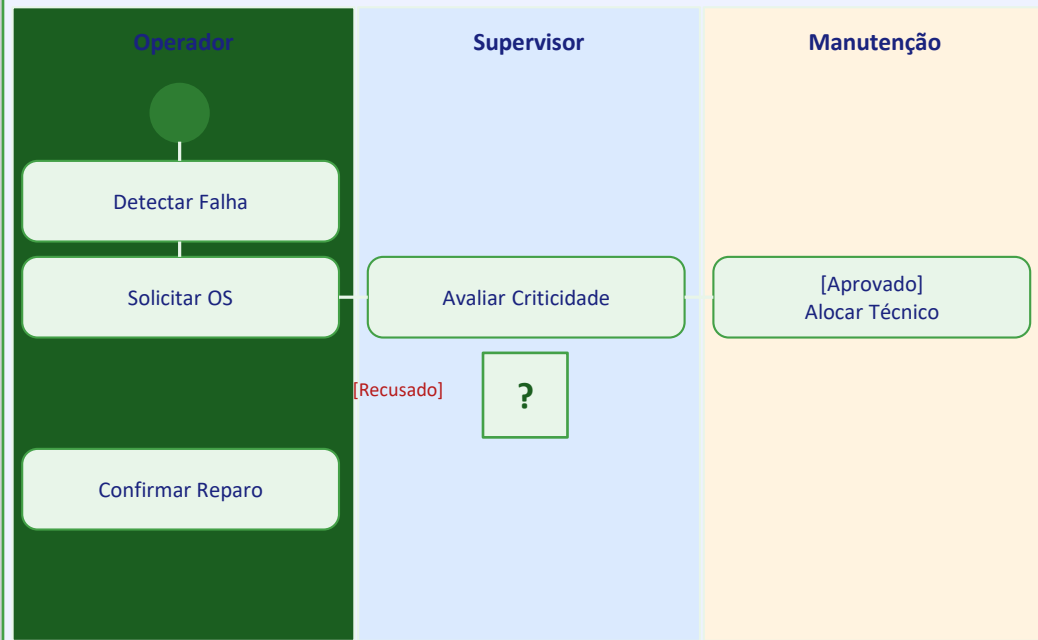
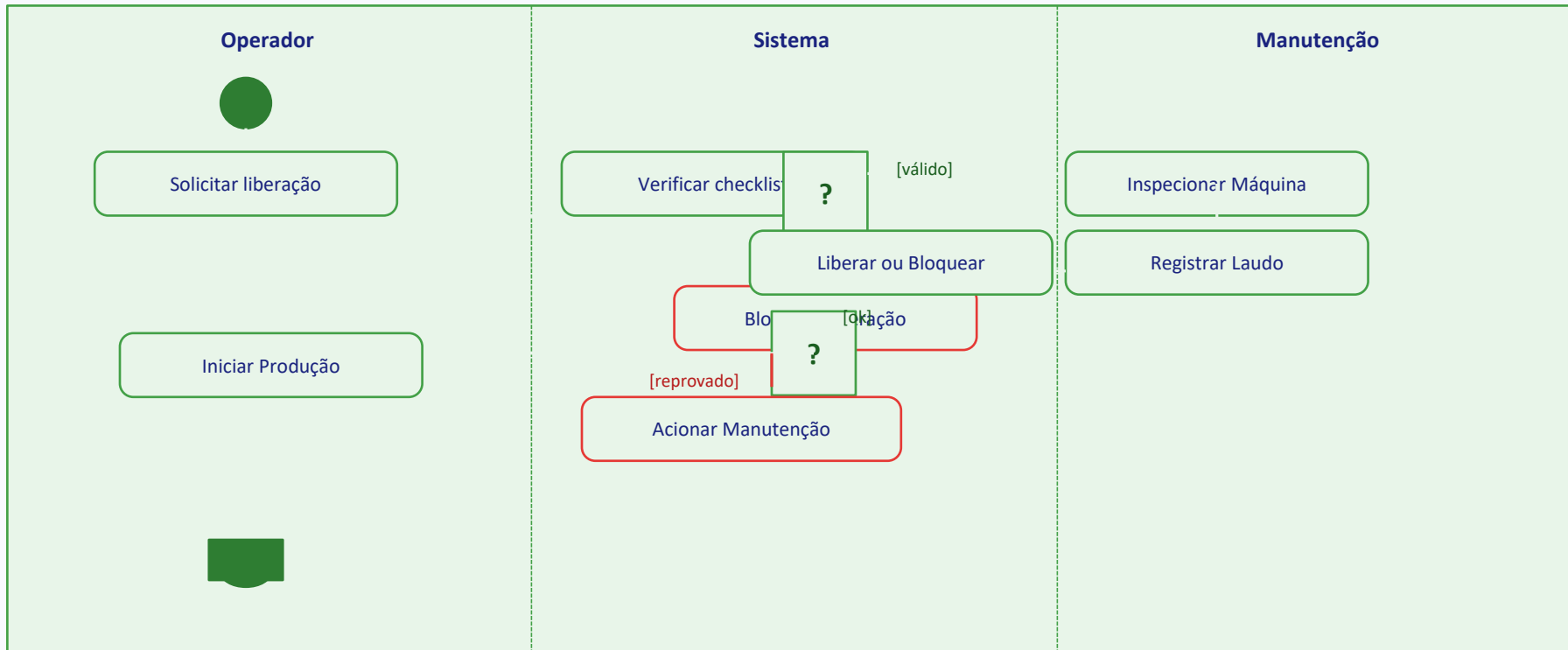


Diagrama de Atividades — Exemplo: Liberação de Equipamento para Produção

Fluxo com swimlanes, decisões e tratamento de exceção para liberação segura de máquina no chão de fábrica.



DER — Diagrama Entidade-Relacionamento

Modela a estrutura de dados. Base para criação do banco de dados relacional. Criado por Peter Chen (1976).

Conceitos Fundamentais

Entidade

Objeto do mundo real com existência independente. Ex.: Aluno, Produto, Pedido.

Atributo

Propriedade ou característica de uma entidade. Ex.: nome, CPF, preço.

Atributo-Chave

Identifica unicamente cada instância. Sublinhado no DER. Ex.: CPF, matrícula.

Relacionamento

Associação entre entidades. Representado por losango com nome.

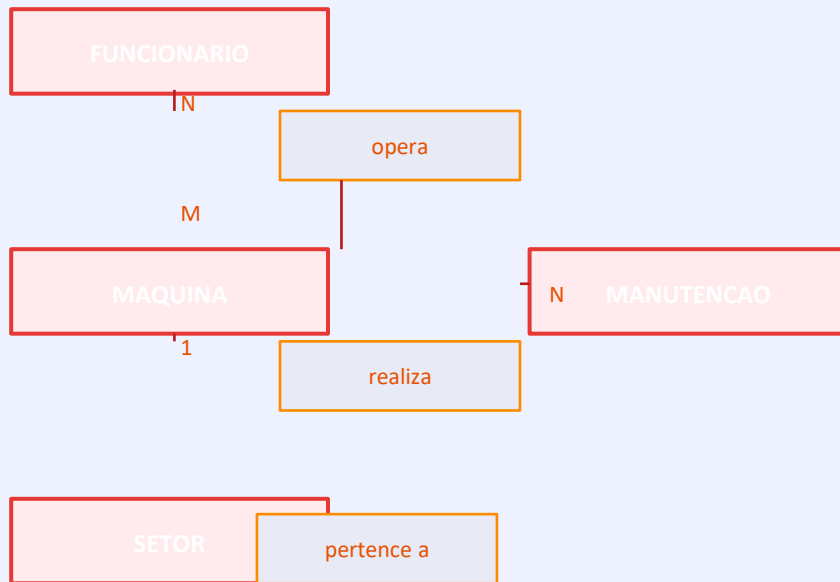
Cardinalidade

Quantidade de instâncias envolvidas: 1:1, 1:N, N:M.

Entidade Fraca

Depende de outra entidade para existir. Retângulo duplo no DER.

Exemplo: Fábrica Industrial



DER — Cardinalidade e Notações

A cardinalidade define quantas instâncias de uma entidade se associam a outra. Existem diferentes notações.

Tipos de Cardinalidade

1 : 1

Uma Pessoa tem exatamente um CPF

1 : N

Um Cliente faz vários Pedidos

N : M

Alunos cursam várias Disciplinas e vice-versa

0 : 1

Uma Pessoa pode ter ou não um Passaporte

Notação de Chen (1976)

Usa retângulos para entidades, elipses para atributos e losangos para relacionamentos. A cardinalidade é indicada com os números 1, N ou M diretamente nas linhas de associação.

Mais usada no ensino e em projetos acadêmicos.

Notação Pé-de-Galinha (Crow's Foot)

Usa símbolos nas extremidades das linhas para indicar cardinalidade: traço (1), círculo (0) e garfo (muitos). É a notação mais usada em ferramentas profissionais como MySQL Workbench, Lucidchart e Astah.

Mais usada em projetos reais e em ferramentas CASE.

DER — Atributos: Tipos e Classificações

Atributos descrevem as propriedades das entidades. Existem diferentes tipos com representações específicas.

Atributo Simples

Representação: Elipse simples

Ex: nome, preço, data

Valor atômico, não divisível. Tipo mais comum.

Atributo Composto

Representação: Elipse com sub-elipses

Ex: endereço (rua, num, CEP)

Pode ser subdividido em partes menores.

Atributo Multivalorado

Representação: Elipse dupla

Ex: telefones, emails

Pode ter vários valores para uma mesma entidade.

Atributo Derivado

Representação: Elipse tracejada

Ex: idade (derivada de dataNasc)

Calculado a partir de outro atributo.

Atributo Chave

Representação: Elipse com texto sublinhado

Ex: CPF, matrícula, CNPJ

Identifica unicamente cada instância da entidade.

Atributo Chave Parcial

Representação: Elipse com texto tracejado

Ex: numItem em entidade fraca

Identifica entidades fracas junto à chave do pai.

DER — Exemplo Completo: Sistema de Manutenção Industrial

Modelo DER completo com entidades, atributos-chave, relacionamentos e cardinalidades de uma fábrica.

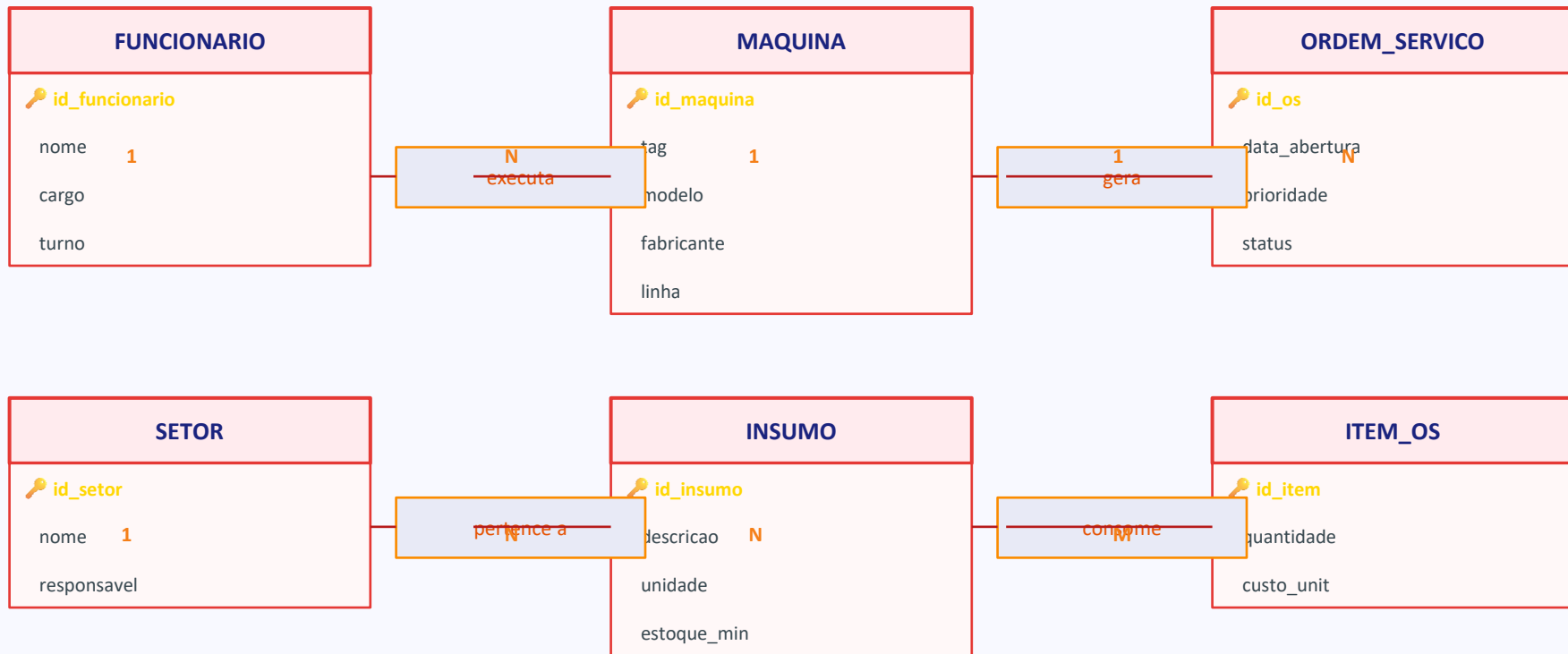


Diagrama de Sequência — UML

Mostra a interação entre objetos ao longo do tempo. Essencial para modelar fluxos de chamadas e protocolos.

Conceitos-Chave

Objeto/Participante

Caixa no topo com nome sublinhado.

Linha de Vida

Linha vertical pontilhada abaixo do objeto.

Ativação

Retângulo estreito: objeto está executando.

Mensagem Síncrona

Seta cheia: chamada aguarda resposta.

Mensagem Assíncrona

Seta aberta: chamada não aguarda retorno.

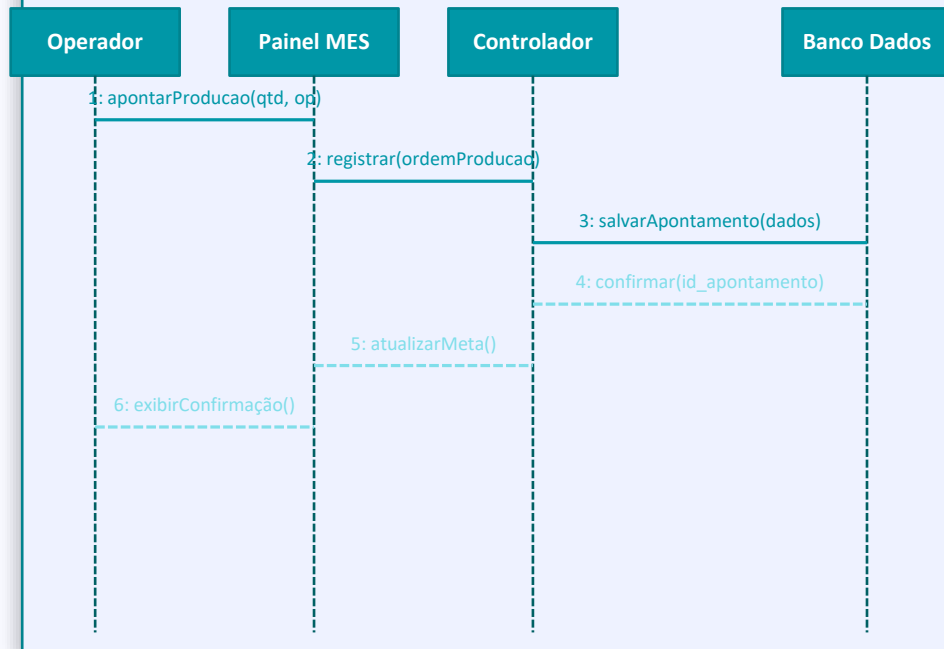
Retorno

Seta tracejada: resultado da chamada.

Alt / Opt / Loop

Fragmentos combinados para condicionais.

Exemplo: Registro de Produção no Sistema MES



BPMN — Business Process Model and Notation

Notação padrão OMG para modelagem de processos de negócio. Versão atual: BPMN 2.0.

Eventos

- Início (círculo simples)
- Intermediário (círculo duplo)
- Fim (círculo preenchido)
- Tipos: Mensagem, Timer, Erro, Sinal

Atividades

- Tarefa (Task): trabalho atômico
- Sub-processo expansível
- Marcadores: Manual, Script, Serviço
- Chamada (Call Activity): reutilização

Gateways

- Exclusivo (X): apenas um caminho
- Paralelo (+): todos os caminhos
- Inclusivo (O): um ou mais caminhos
- Baseado em Eventos (hexágono)

Raias e Artefatos

- Pool: processo de um participante
- Lane: subdivide responsabilidades
- Objeto de Dados: entrada/saída
- Anotações: comentários no modelo

Ferramentas de Modelagem — Parte 1

Ferramentas especializadas que suportam criação, edição e exportação de diagramas padronizados.

Astah Community

UML Completo | Gratuito

Suporta todos os principais diagramas UML 2.x. Versão Community gratuita para estudantes e educadores. Interface intuitiva, amplamente usada no ensino técnico e universitário.

Destaques:

- Gratuito para uso acadêmico
- Todos os diagramas UML
- Exporta PNG, PDF, SVG

draw.io / diagrams.net

Diagramas Gerais | Gratuito

Ferramenta online e offline, integrada ao Google Drive e VSCode. Suporta UML, BPMN, fluxogramas, redes. Formato XML editável e exportação em múltiplos formatos.

Destaques:

- 100% gratuito e open source
- Funciona no navegador sem instalação
- Integração com Google Drive e GitHub

StarUML

UML | Pago (trial)

Aplicação desktop leve e moderna para macOS, Windows e Linux. Suporte a todos os diagramas UML 2.x com sintaxe validada. Popular entre desenvolvedores.

Destaques:

- Interface moderna e ágil
- Validação de sintaxe UML
- Extensível via plugins Node.js

Ferramentas de Modelagem — Parte 2

Lucidchart

Colaborativo | Freemium

Plataforma web com foco em colaboração em tempo real. Suporta UML, BPMN, wireframes e mapas mentais. Muito usada em empresas para trabalho em equipe.

Destaques:

- Colaboração em tempo real
- Templates profissionais prontos
- Integração com Slack, Jira, Confluence

Enterprise Architect

UML Avançado | Pago

Ferramenta robusta da Sparx Systems para projetos empresariais complexos. Suporta engenharia reversa (código → diagrama) e geração de código a partir de modelos UML.

Destaques:

- Engenharia reversa de código
- Geração automática de documentação
- Gestão de requisitos integrada

Bizagi Modeler

BPMN | Gratuito

Especializada em BPMN 2.0. Ferramenta gratuita para modelagem de processos. Exporta para PDF, Word e Web. Muito adotada em empresas para mapeamento de processos.

Destaques:

- 100% focada em BPMN 2.0
- Exporta em múltiplos formatos
- Simulação de processos incluída

Comparativo de Ferramentas

Ferramenta	UML	BPMN	DER	Colaboração	Preço	Nível
Astah Community	✓	✗	Parcial	✗	Gratuito	Iniciante/Acadêmico
draw.io	Básico	✓	✓	✓	Gratuito	Todos os Níveis
StarUML	✓	✗	✗	✗	Pago	Intermediário
Lucidchart	✓	✓	✓	✓	Freemium	Intermediário/Empresa
Enterprise Arch.	✓	✓	✓	Parcial	Pago	Avançado/Empresa
Bizagi Modeler	✗	✓	✗	Parcial	Gratuito	Todos os Níveis

Como Escolher a Técnica e a Ferramenta Certa?

A escolha depende do objetivo, do público e do contexto do projeto.

? Preciso mostrar
a estrutura do código?

✓ Diagrama de Classes (UML)

✂ Astah / StarUML

? Preciso mapear
um processo de negócio?

✓ BPMN 2.0

✂ Bizagi / draw.io

? Preciso projetar
o banco de dados?

✓ DER (Chen ou Crow's Foot)

✂ draw.io / MySQL Workbench

? Preciso documentar
um fluxo de interação?

✓ Diagrama de Sequência (UML)

✂ Astah / Lucidchart

? Preciso mostrar
o que o sistema faz?

✓ Casos de Uso (UML)

✂ Astah / draw.io

? Preciso modelar
um fluxo de trabalho?

✓ Diagrama de Atividades (UML)

✂ draw.io / Lucidchart

Boas Práticas x Erros Comuns na Modelagem

✓ Boas Práticas

- Escolha a notação adequada ao contexto e ao público
- Mantenha os diagramas simples e legíveis (KISS)
- Documente os modelos junto ao repositório de código
- Valide os modelos com toda a equipe antes de prosseguir
- Use nomenclatura clara, consistente e padronizada
- Revise e atualize os diagramas a cada iteração/sprint
- Comece pelo diagrama mais simples para o contexto

✗ Erros Comuns

- Modelar apenas para cumprir uma etapa burocrática
- Criar diagramas complexos demais sem necessidade real
- Não atualizar modelos após mudanças no sistema
- Misturar notações UML, BPMN e fluxograma incorretamente
- Ignorar os diagramas após a fase de análise e design
- Usar ferramentas sem conhecer a notação que suportam
- Não envolver o cliente ou usuário final na validação

O que o Mercado Exige — Conclusão

Profissionais de desenvolvimento de sistemas que dominam modelagem são mais valorizados e produtivos.



BPMN é essencial para trabalhar com analistas de negócio, produto e processos em empresas.



Ferramentas como Astah, draw.io e Bizagi são exigidas em processos seletivos e projetos reais.



A integração entre técnicas (UML + BPMN + DER) forma o perfil completo de um analista de sistemas.



Modelos bem feitos economizam tempo, dinheiro e evitam retrabalho ao longo de todo o projeto.

Desenvolvedores que sabem modelar constroem sistemas mais confiáveis, com menos retrabalho e mais qualidade.